

Polymorphism in C# Method Hiding



Mr. Bangar Raju



Understanding Method Hiding and Overriding in C#

By Albert Harmon Published July 29, 2017 ⌚ 4 min read [Education](#)



TLDR This blog post explains the concepts of method hiding and method overriding in C#, highlighting their differences, practical implementations, and how to call parent class methods from child classes.

In this tutorial, we will explore two important concepts in C#: method hiding and method overriding. Both concepts are essential for object-oriented programming and understanding how inheritance works in C#. Let's dive into the definitions, differences, and practical implementations of these concepts.

What is Method Overriding?

Method overriding is a technique that allows a child class to provide a specific implementation of a method that is already defined in its parent class. The overriding method in the child class must have the same name and signature as the method in the parent class. To enable overriding, the method in the parent class must be declared as ``virtual``.

Example of Method Overriding

Consider the following example:

```
public class ParentClass
{
    public virtual void TestOne()
    {
        Console.WriteLine("Method TestOne from Parent Class");
    }
}
```

```
public class ChildClass : ParentClass
{
    public override void TestOne()
    {
        Console.WriteLine("Method TestOne from Child Class");
    }
}
```

In this example, `TestOne` is a virtual method in the `ParentClass`, and it is overridden in the `ChildClass`. When you call `TestOne` on an instance of `ChildClass`, it will execute the overridden method.

What is Method Hiding?

Method hiding, also known as shadowing, allows a child class to define a method with the same name as a method in the parent class, even if the method in the parent class is not declared as `virtual`. This means that the child class can hide the parent class method without any permission.

Example of Method Hiding

Here's how method hiding works:

```
public class ParentClass
{
    public void TestTwo()
    {
        Console.WriteLine("Method TestTwo from Parent Class");
    }
}

public class ChildClass : ParentClass
{
    public new void TestTwo()
    {
        Console.WriteLine("Method TestTwo from Child Class");
    }
}
```

In this example, `TestTwo` in the `ChildClass` hides the `TestTwo` method from the `ParentClass`. The `new` keyword is used to indicate that this method is hiding the base class method.

Key Differences Between Method Overriding and Method Hiding

1. Permission:

- In method overriding, the parent class gives permission to the child class to override its method by declaring it as ``virtual``.
- In method hiding, the child class can hide a method from the parent class without any permission.

2. Method Accessibility:

- Overridden methods can be accessed through a parent class reference if the method is overridden in the child class.
- Hidden methods cannot be accessed through a parent class reference; they are considered pure child class members.

3. Usage of Keywords:

- The ``override`` keyword is used in method overriding to indicate that a method is overriding a base class method.
- The ``new`` keyword is used in method hiding to indicate that a method is hiding a base class method.

Calling Parent Class Methods from Child Class

After implementing method hiding or overriding, you might want to call the parent class methods from the child class. This can be done in two ways:

Option 1: Creating an Instance of the Parent Class

You can create an instance of the parent class within the child class and call its methods:

```
ParentClass p = new ParentClass();  
p.TestOne(); // Calls the parent class method
```

Option 2: Using the `base` Keyword

You can also use the `base` keyword to call the parent class methods directly:

```
public void CallParentMethods()  
{  
    base.TestOne(); // Calls the parent class method  
}
```

Conclusion

In summary, method overriding and method hiding are two fundamental concepts in C# that allow for flexible and powerful object-oriented programming. Understanding the differences between these two methods is crucial for effective inheritance and method management in your applications. By using these techniques, you can create more dynamic and maintainable code.

Thank you for reading! For more tutorials and insights, stay tuned to our channel.